



Chương 3.

PHÂN TÍCH HỆ THỐNG

Mô hình hóa hành vi

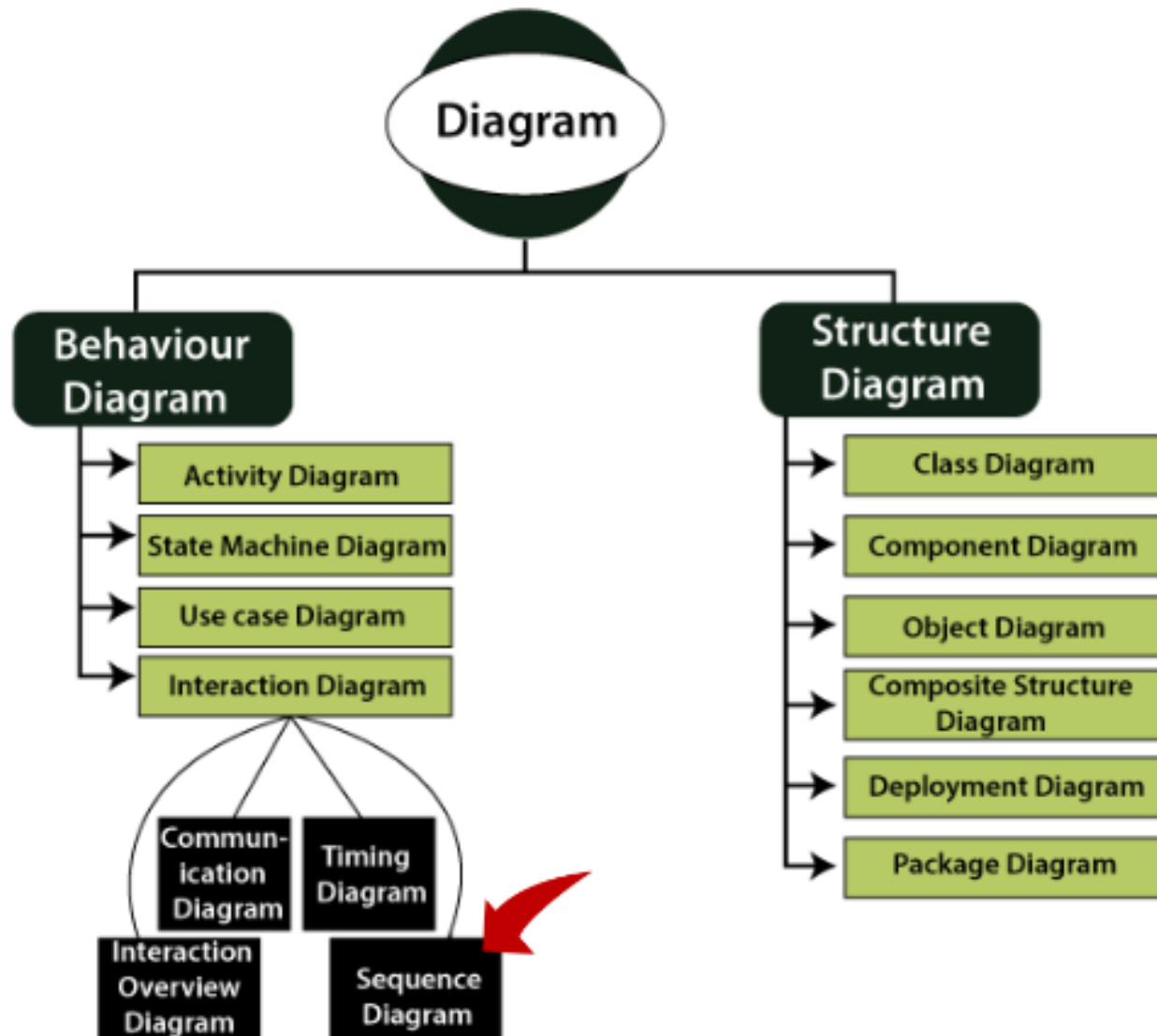
Giới thiệu

Sơ đồ tuần tự - Sequence Diagram

Sơ đồ giao tiếp - Communication Diagram

Sơ đồ máy trạng thái - State Diagram

Giới thiệu



Interaction Diagram

Tương tác (Interaction): là hành vi được mô tả bằng chuỗi thông điệp trao đổi giữa các đối tượng trong một khung cảnh **tương tác** được xác định bởi một **cộng tác**:

- Mô hình hóa phương diện động của hệ thống, mô tả tương tác giữa các đối tượng
- Thường được đề mô tả kịch bản của ca sử dụng (Use Case)

Cộng tác

- Các đối tượng cần phải cộng tác với nhau
 - Các đối tượng sẽ trở nên vô nghĩa nếu chúng không cộng tác với nhau để giải quyết vấn đề:
 - Mỗi đối tượng có trách nhiệm quản lý hành vi và trạng thái
 - Không một ai, không một đối tượng nào làm được mọi việc
- ➔ **Cộng tác (collaboration):** là một dạng **tương tác** nhưng mang tính chất **hợp tác** và có mục tiêu chung để đạt được kết quả nhất định

Interaction Diagram (tt)

■ Các loại sơ đồ tương tác

- Sơ đồ tuần tự (Sequence diagram)
- Sơ đồ giao tiếp (Communication diagram)
- Sơ đồ thời gian (Timing Diagram)
- Sơ đồ tương tác tổng quát (Interaction Overview diagram)

Sequence Diagram

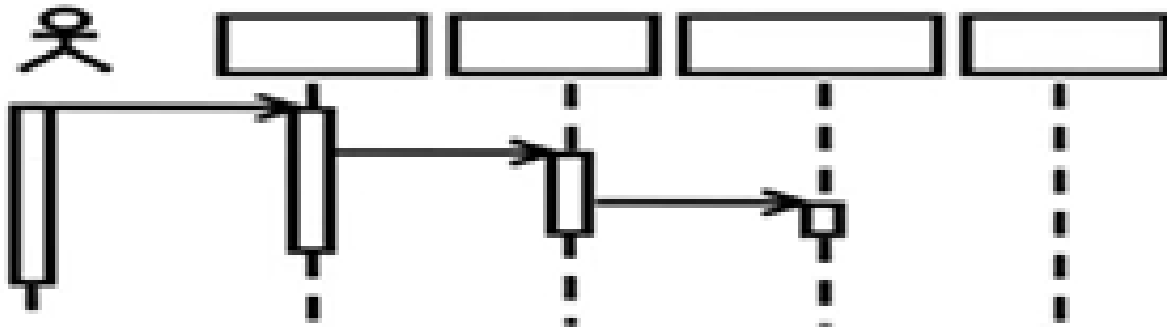
- Sơ đồ tuần tự mô hình hóa **tuần tự** các **thông điệp** tương tác giữa các đối tượng theo thời gian khi thực hiện một Use Case
- Được sử dụng để xác định và chỉ rõ vai trò của các đối tượng tham gia vào luồng sự kiện của Use Case
- Các đối tượng tham gia vào hoạt động được liệt kê từ trái sang phải tùy theo thời điểm tham gia vào chuỗi thông điệp
- Sơ đồ tuần tự mô tả các đối tượng và lớp liên quan đến kịch bản và chuỗi thông điệp được trao đổi giữa các đối tượng cần thiết để thực hiện chức năng của kịch bản trong đặc tả Use Case

Sequence Diagram (tt)

Sơ đồ tuần tự chỉ ra:

- Các đối tượng tham gia và tương tác
- Thời gian sống của các đối tượng
- Trình tự các thông điệp được trao đổi

➔ Đọc sơ đồ tuần tự từ đỉnh xuống đáy

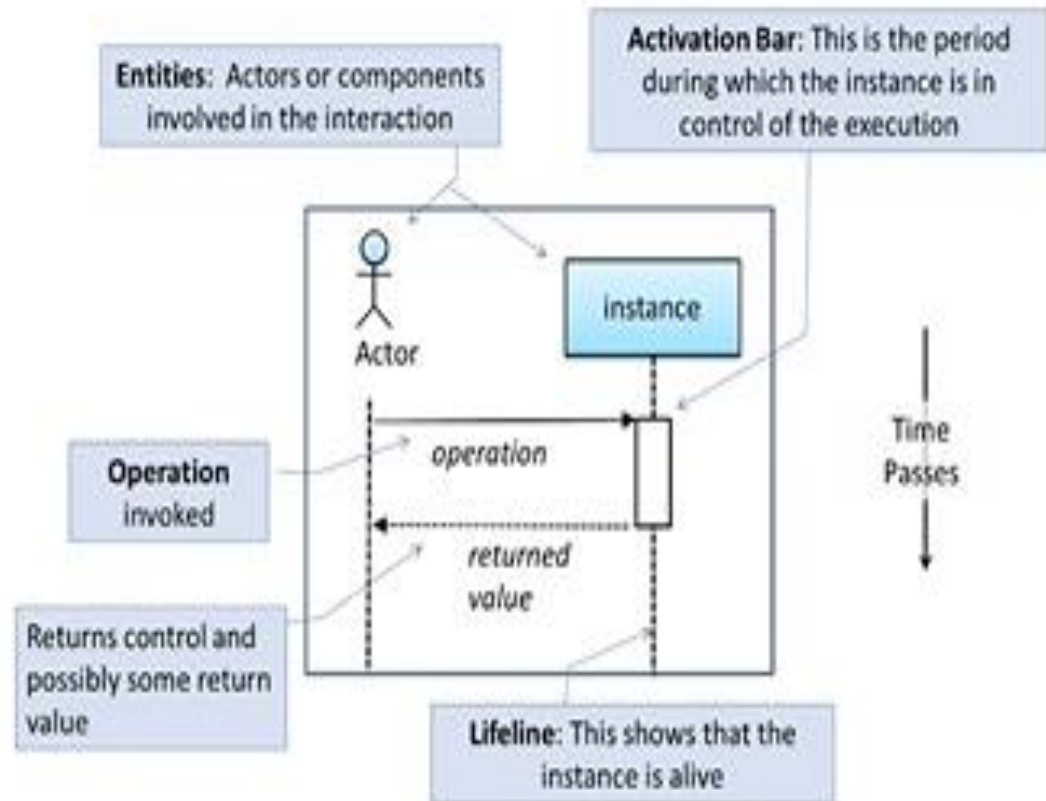


Ứng dụng Sequence Diagram

- **Thiết kế hệ thống:** Giúp hình dung và thiết kế các tương tác giữa các thành phần hệ thống, nó đảm bảo tính logic và hiệu quả cho hệ thống
- **Lập trình:** Biểu đồ này cũng cung cấp hướng dẫn chi tiết cho việc triển khai code, giúp lập trình viên dễ dàng nắm bắt logic và thực hiện các chức năng
- **Kiểm tra và bảo trì:** Giúp xác định lỗi tiềm ẩn trong thiết kế và code, hỗ trợ việc kiểm tra và bảo trì hệ thống hiệu quả

Các thành phần trong Sequence Diagram

1. Lifeline
2. Activation Bar
3. Message
4. Combined Fragment



1. Lifeline

- Một sơ đồ tuần tự được tạo thành từ một ký hiệu lifeline
- Sắp xếp theo chiều ngang ở phía trên của sơ đồ
- Lifeline không được chồng chéo lên nhau
- Đại diện cho các đối tượng hoặc các bộ phận khác nhau trong hệ thống tương tác với nhau khi thực hiện một Use Case
- Hình chữ nhật trên đầu lifeline mô tả một đối tượng trong hệ thống
- Trục tọa độ, trục ngang thể hiện các đối tượng, trục đứng thể hiện thời gian

1. Lifeline (tt)

Một đối tượng có thể đặt tên theo một trong 3 cách:

- Tên đối tượng → SV1
- Tên đối tượng và lớp của nó → SV1: SinhVien
- Chỉ tên lớp (đối tượng ẩn danh) → : SinhVien

SV1



SV1: SinhVien

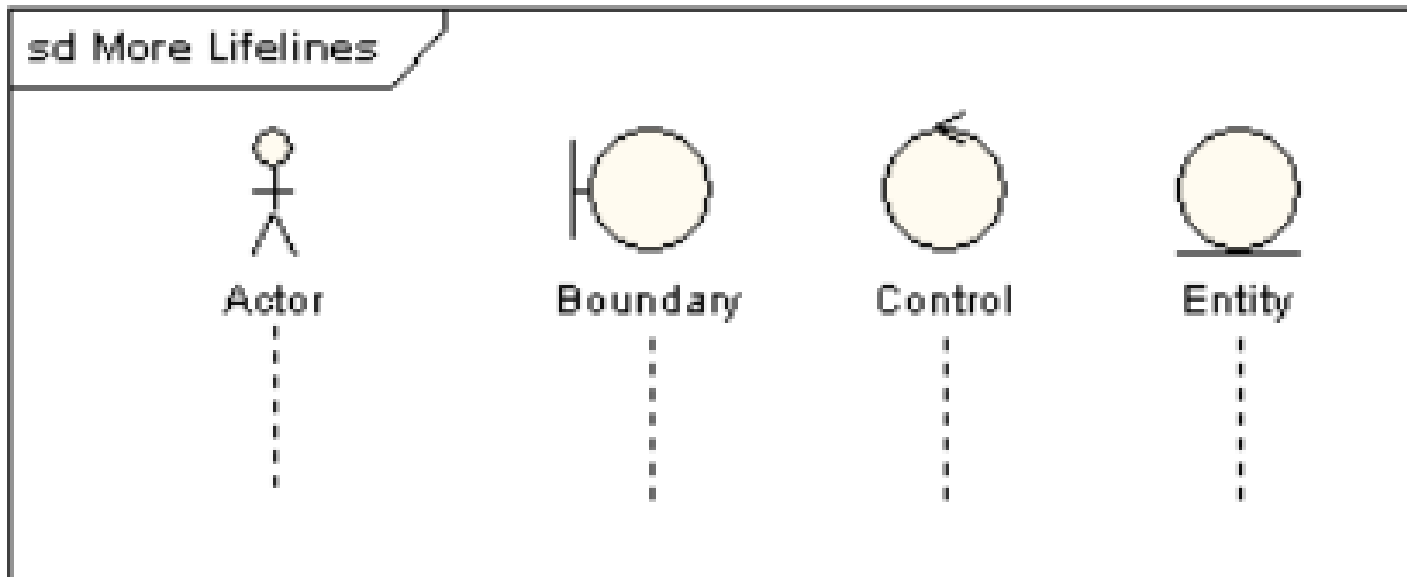


: SinhVien



1. Lifeline (tt)

- Các đối tượng sẽ chịu trách nhiệm những công việc như quản lý, di chuyển, phản hồi các yêu cầu trong hệ thống
- Các đối tượng hoạt động cùng nhau bằng cách giao tiếp hoặc tương tác với nhau



1. Lifeline (tt)

Các loại Lifeline

- **Lifeline actor:** biểu diễn đối tượng (thường là người dùng hoặc hệ thống bên ngoài) thực hiện Use Case, tương tác trực tiếp với hệ thống
- **Lifeline boundary:** biểu diễn ranh giới của hệ thống, thường là giao diện (UI) hoặc điểm tiếp xúc giữa hệ thống và các tác nhân bên ngoài
- **Lifeline Control:** biểu diễn đối tượng điều khiển, đóng vai trò trung gian, xử lý logic và điều phối giữa Boundary và Entity
- **Lifeline Entity:** biểu diễn dữ liệu hoặc đối tượng lưu trữ trong hệ thống, chẳng hạn như cơ sở dữ liệu hoặc thực thể nghiệp vụ

1. Lifeline (tt)

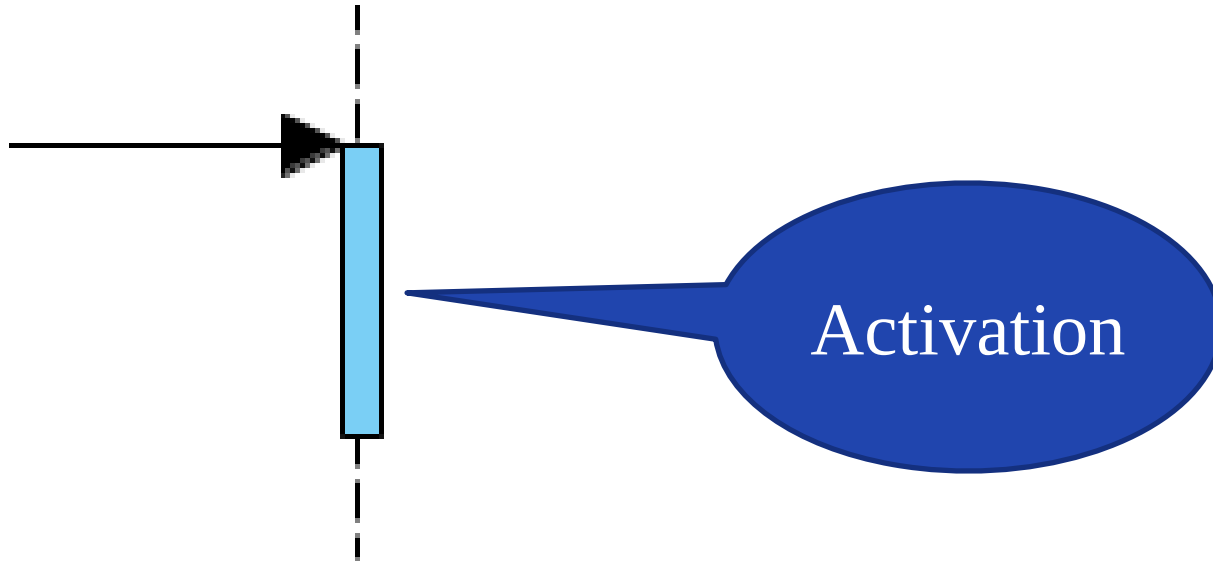
Ví dụ: Một khách hàng (Actor) muốn đặt hàng qua giao diện website (Boundary). Hệ thống xử lý yêu cầu đặt hàng (Control) và lưu thông tin đơn hàng vào cơ sở dữ liệu (Entity)

Mô tả các Lifeline:

- **Actor:** Khách hàng (người dùng thực hiện việc đặt hàng)
- **Boundary:** Giao diện website (nơi khách hàng nhập thông tin đơn hàng)
- **Control:** Bộ điều khiển đơn hàng (xử lý logic đặt hàng, kiểm tra thông tin)
- **Entity:** cơ sở dữ liệu đơn hàng (lưu trữ thông tin đơn hàng)

2. Activation Bar

Được sử dụng để mô tả thời gian cần để thực thi một hành động nào đó. Được tạo trong chu kỳ sống của một đối tượng



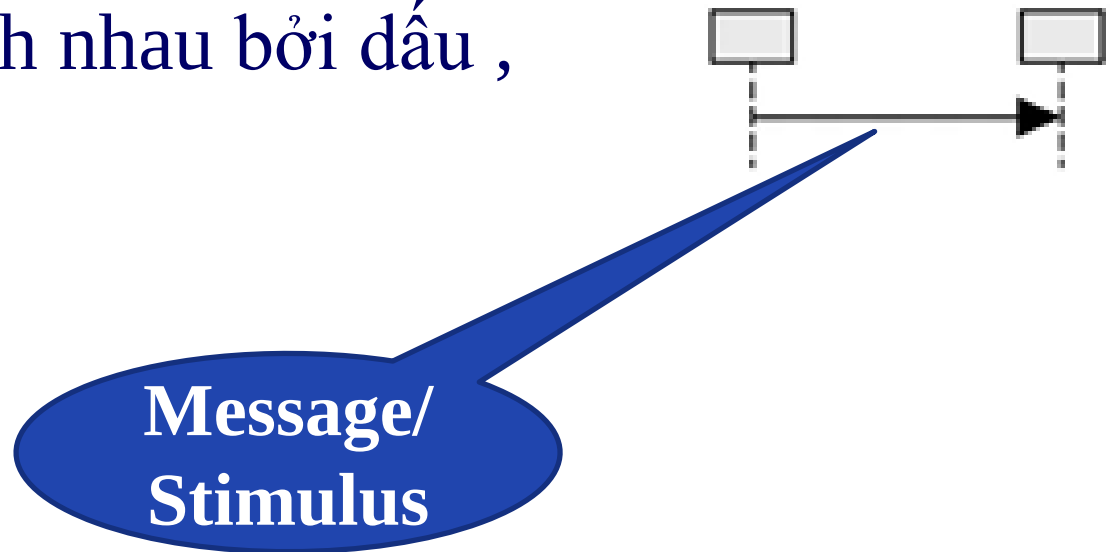
3. Message

- Message/ Stimulus
- Async Message
- Self Message
- Reply or Return Message
- Create Message, Delete Message
- Found Message, Lost Message

Thông điệp (message) là sự giao tiếp một chiều giữa hai đối tượng ở dạng luồng điều khiển cùng với các thông tin đính kèm từ đối tượng gửi đến đối tượng nhận.

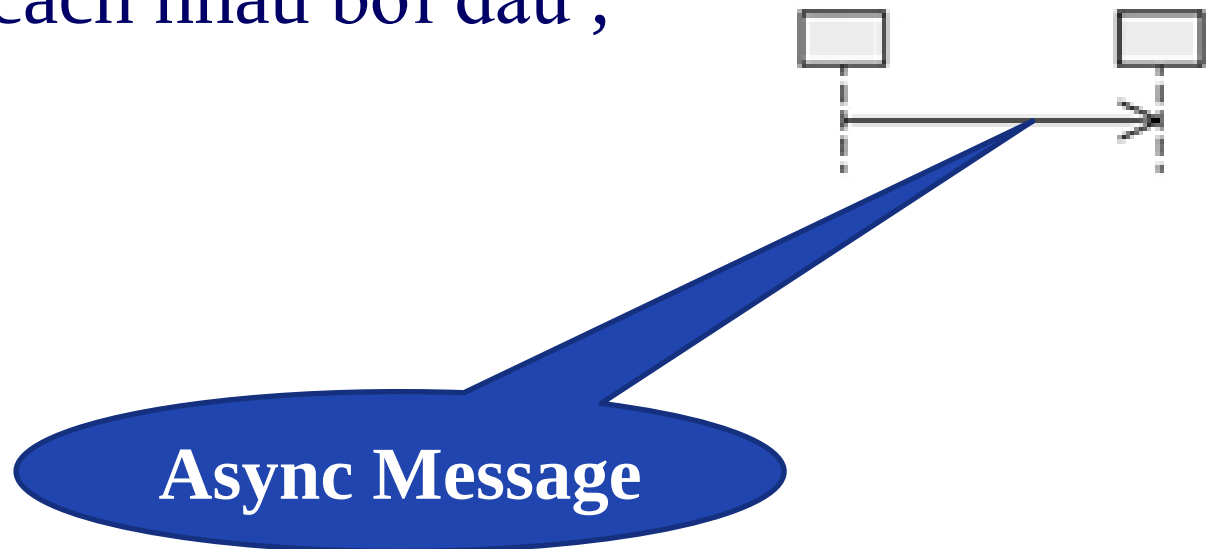
3. Message (tt)

- **Message/ Stimulus:** Thông điệp **đồng bộ**, là sự giao tiếp hai chiều, đối tượng gửi thông điệp sẽ chờ đối tượng nhận thông điệp phản hồi và sẽ không gửi thông điệp khác cho đến khi nhận được phản hồi
- **Cú pháp của thông điệp:** msg (par1, par2)
msg: tên thông điệp
par: tham số cách nhau bởi dấu ,
- ✓ **Ký hiệu:**



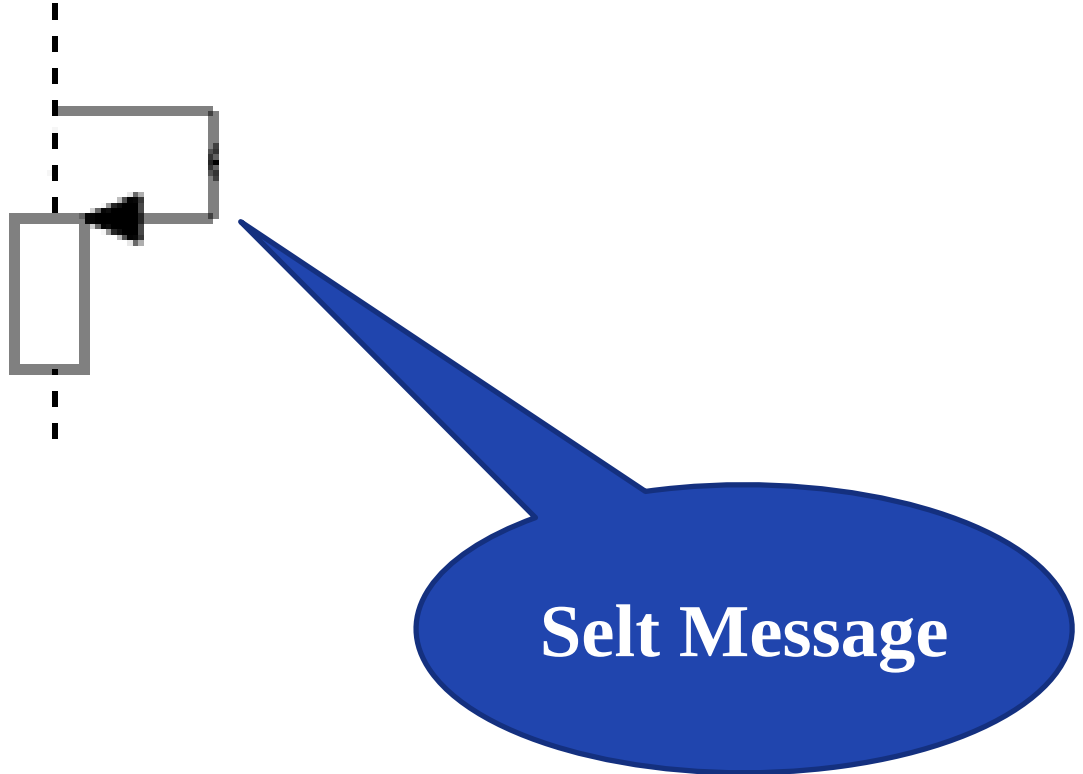
3. Message (tt)

- **Async Message:** thông điệp **không đồng bộ**, là sự giao tiếp một chiều, tiếp tục gửi thông điệp khác mà không chờ đối tượng nhận thông điệp phản hồi
- **Cú pháp của thông điệp:** msg (par1, par2)
msg: tên thông điệp
par: tham số cách nhau bởi dấu ,
- ✓ **Ký hiệu:**



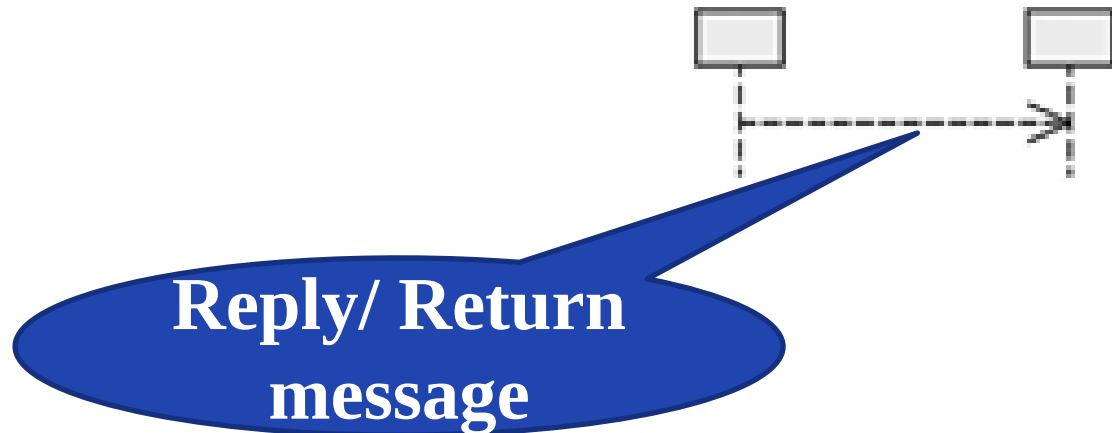
3. Message (tt)

- **Selt Message:** thông điệp **gọi chính nó**, đối tượng gửi thông điệp cho chính nó để thực hiện các hàm nội tại
- ✓ **Ký hiệu:**



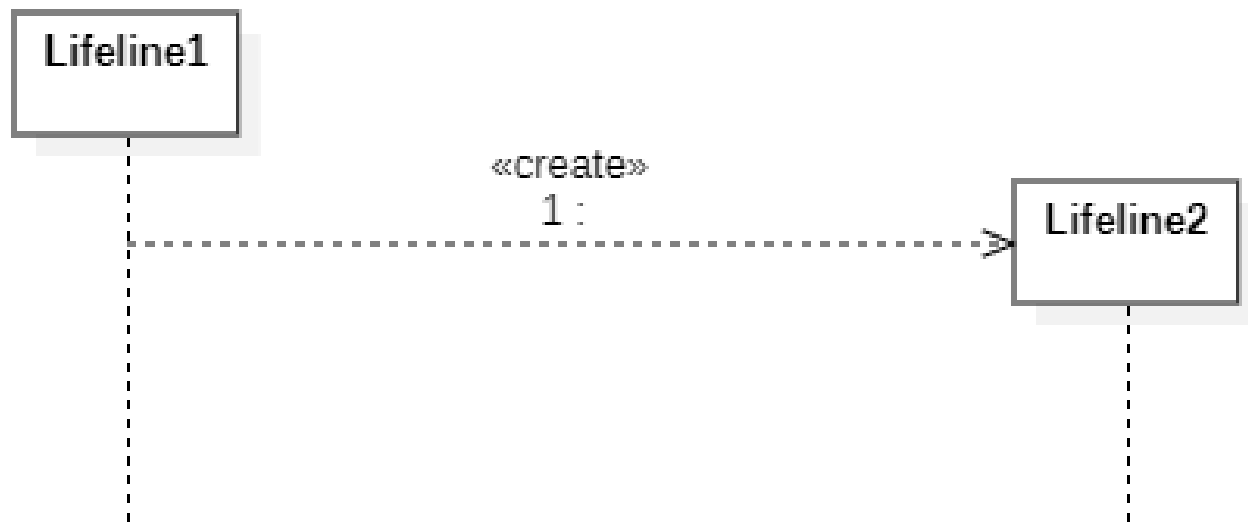
3. Message (tt)

- **Reply/ Return Message:** thông điệp trả lời lại khi nhận được một yêu cầu hoặc sau khi kiểm tra tính đúng đắn của một điều kiện nào đó
- **Cú pháp của thông điệp:** `att = msg (par1, par2): val`
att = giá trị trả về có thể gán cho một biến nào đó
msg: tên thông điệp
par: các tham số cách nhau bởi dấu ,
val: giá trị trả về
- ✓ **Ký hiệu:**



3. Message (tt)

- **Create Message:** thông điệp tạo mới (đối tượng), đường mũi tên nét đứt, đầu mũi tên hướng từ đầu của lifeline của đối tượng được tạo
- ✓ **Ký hiệu:**



3. Message (tt)

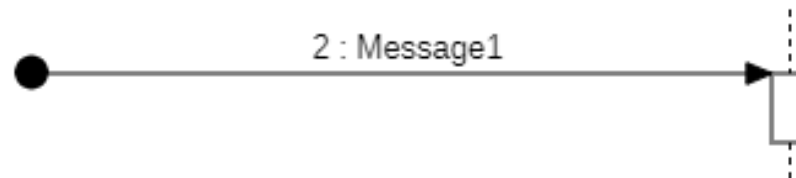
- **Delete Message:** thông điệp hủy (hủy đối tượng), đối tượng bị xóa bỏ, dấu chéo tại điểm cuối của lifeline
- ✓ **Ký hiệu:**



3. Message (tt)

- **Found Message:** thông điệp vô danh không biết đối tượng nào gửi tới, bên gửi thông điệp không biết hoặc không liên quan

✓ **Ký hiệu:**

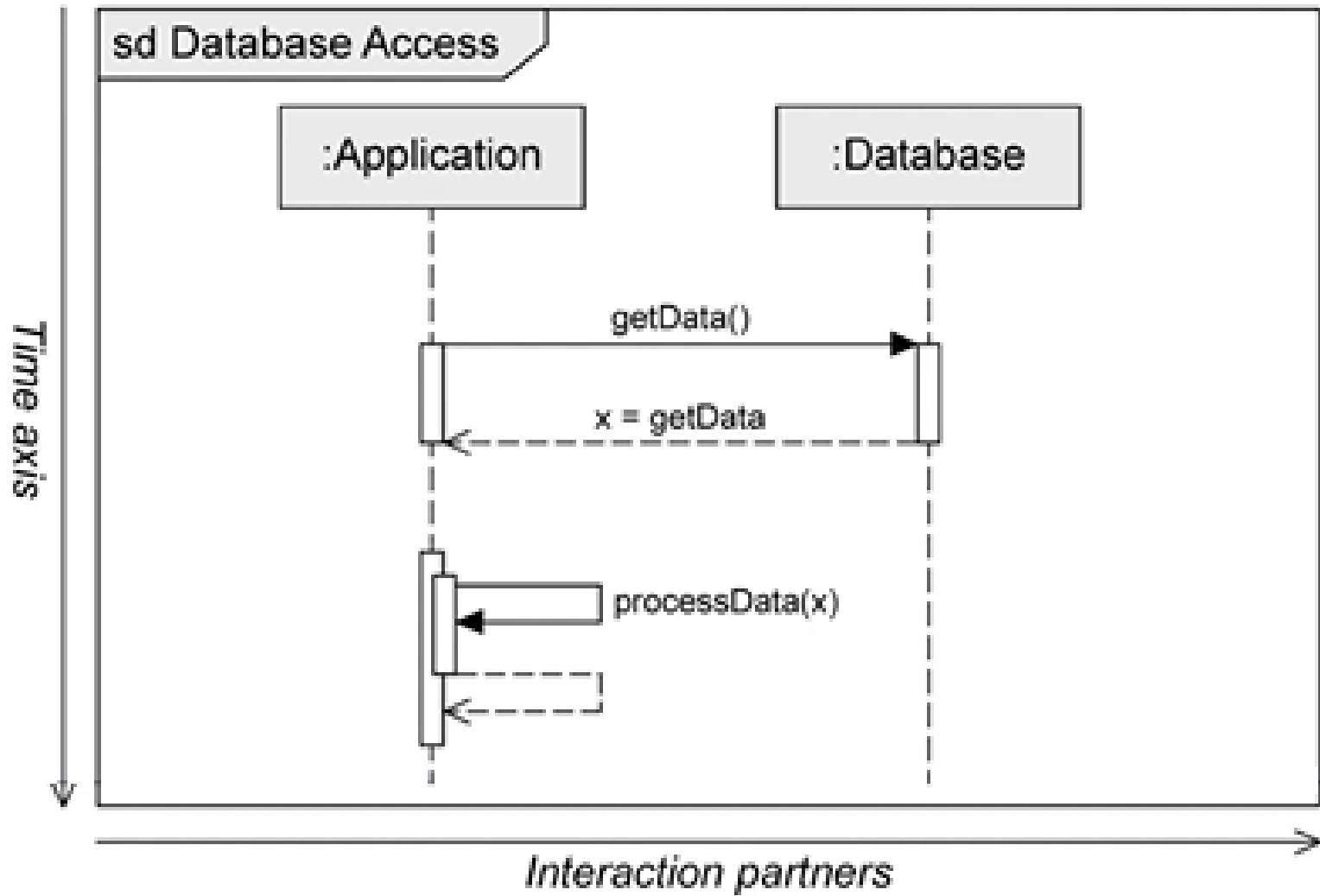


- **Lost Message:** thông điệp bị mất, không biết ai gửi, cũng không biết mình gửi cho ai

✓ **Ký hiệu:**



Ví dụ



Combined fragment

Nhánh và Vòng lặp (Branches and loops)

Toán tử	Mục đích	Giải thích
Alt	Tương tác có thể lựa chọn	Dùng để mô tả các lựa chọn thay thế (như cấu trúc if-else): chỉ một trong các toán hạng (operands) sẽ được thực hiện
Opt	Tương tác tùy chọn	Dùng cho một tương tác tùy chọn (như cấu trúc if đơn): toán hạng chỉ được thực hiện nếu một điều kiện cụ thể là đúng
Loop	Tương tác lặp	Dùng để chỉ ra rằng một tương tác được lặp lại (như cấu trúc for hoặc while)
Break	Tương tác ngoại lệ	Dùng để chỉ ra rằng phần còn lại của biểu đồ trình tự sẽ bị hủy nếu điều kiện trong toán hạng break là đúng

Combined fragment (tt)

Đồng thời và Thứ tự (Concurrency and order)

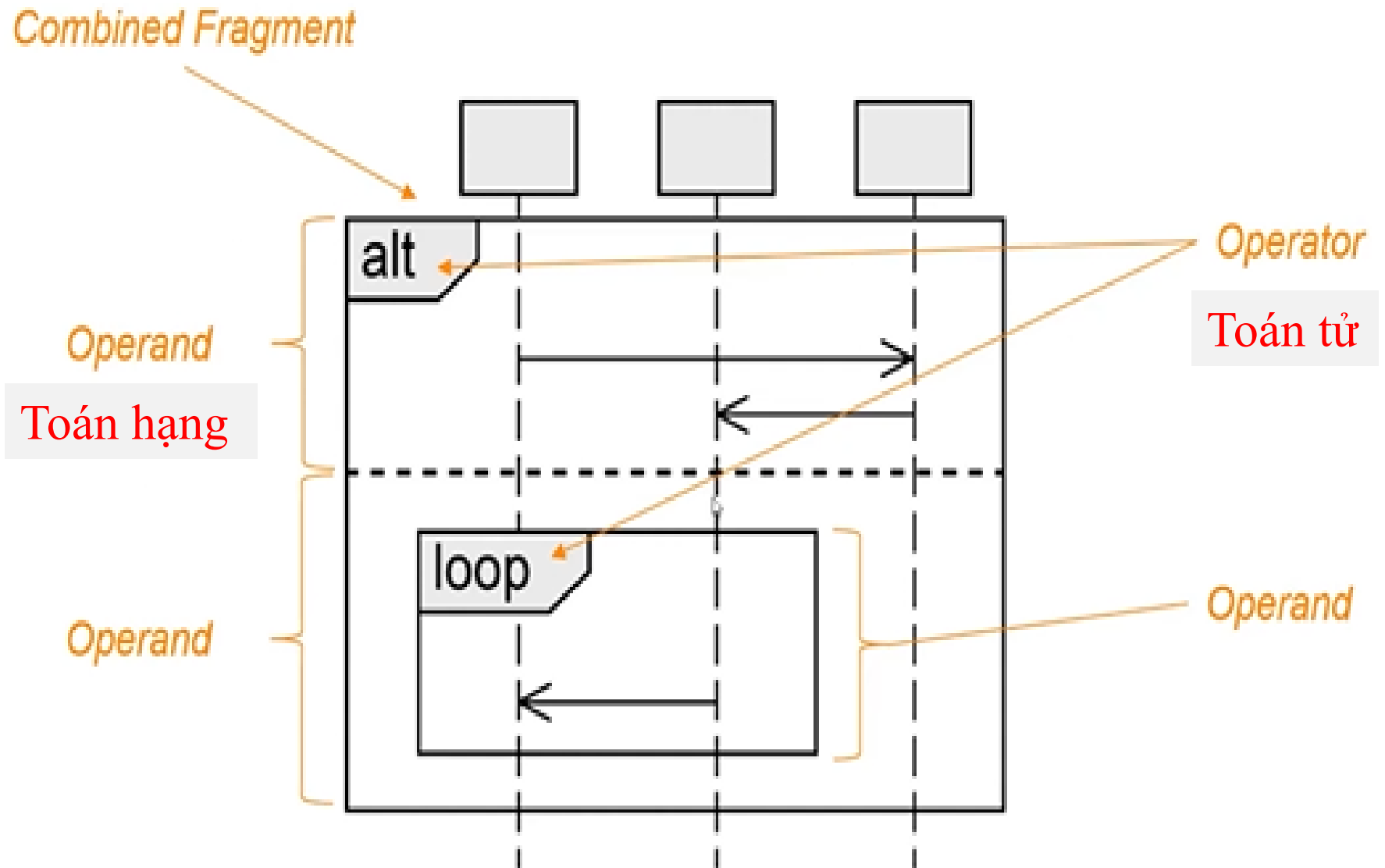
Toán tử	Mục đích	Giải thích
Seq	Lệnh yếu	Các thông điệp trong toán hạng phải được thực hiện theo thứ tự tuần tự đã cho
Strict	Lệnh mạnh	Tương tự seq, nhưng yêu cầu tuân thủ nghiêm ngặt thứ tự của các thông điệp
Par	Tương tác song song	Các thông điệp trong các toán hạng khác nhau có thể được thực hiện đồng thời hoặc xen kẽ bất kỳ lúc nào
Critical	Tương tác đơn nguyên	Một vùng đơn nguyên (atomic) mà các thông điệp bên trong không thể bị ngắt quãng bởi bất kỳ thông điệp nào từ bên ngoài vùng đó

Combined fragment (tt)

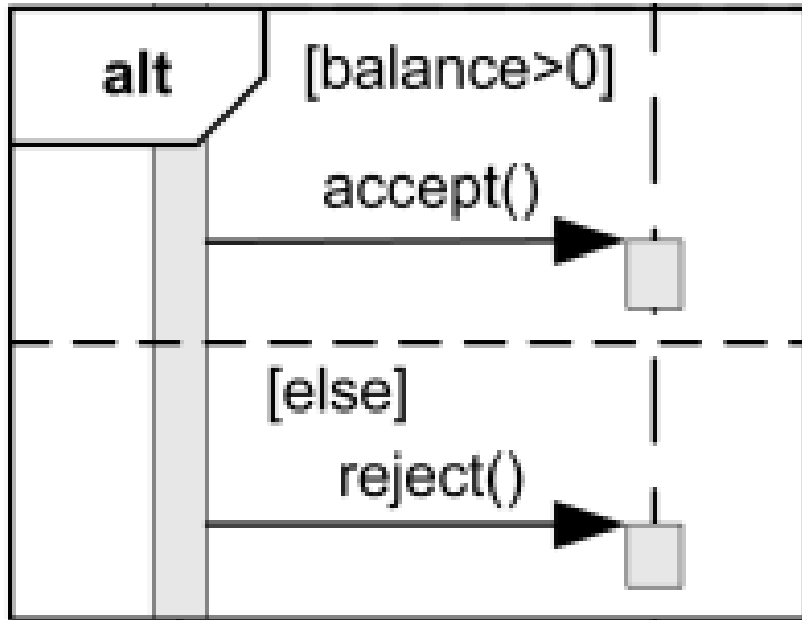
Bộ lọc và Khẳng định (Filters and assertions)

Toán tử	Mục đích	Giải thích
Ignore	Tương tác không phù hợp	Liệt kê các thông điệp có thể xảy ra nhưng không liên quan đến trường hợp đang xét và nên được bỏ qua
Consider	Tương tác phù hợp	Liệt kê các thông điệp liên quan đến trường hợp đang xét; bất kỳ thông điệp nào khác nên được bỏ qua (ngược lại với ignore)
Assert	Tương tác chèn	Khẳng định rằng toán hạng phải là phần duy nhất có thể xảy ra; bất kỳ đường thực thi nào khác là không hợp lệ
Neg	Tương tác không hợp lệ	Mô tả một tương tác mà theo định nghĩa, không được phép xảy ra

Combined fragment (tt)



Alternative



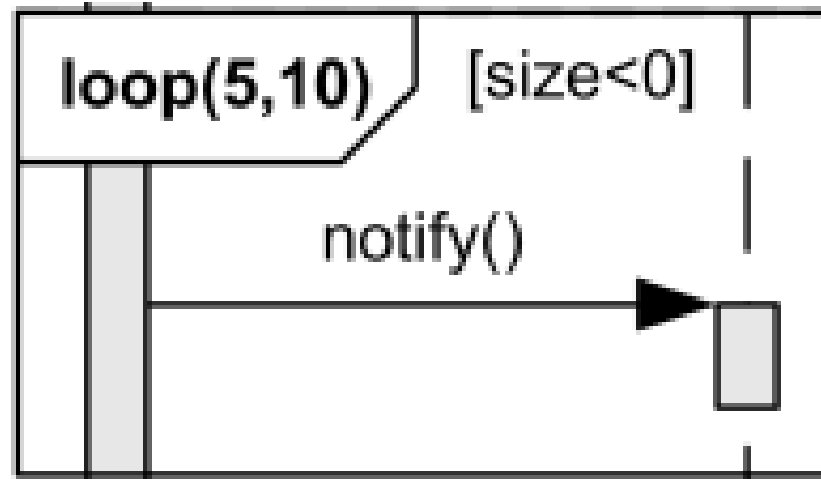
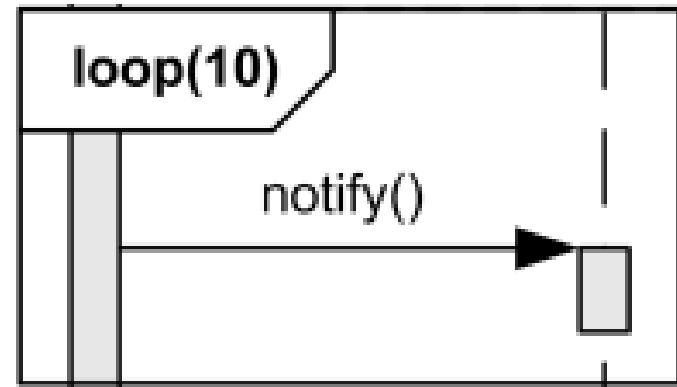
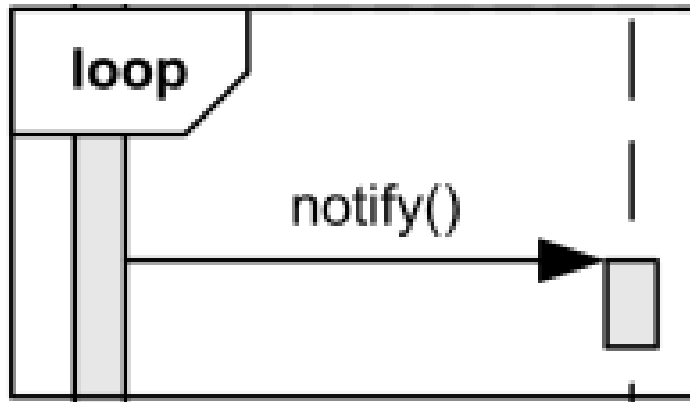
- Alt biểu diễn một lựa chọn hoặc các phương án thay thế về hành vi
- Kiểu cấu trúc if/ else
- Điều kiện được phân tách bằng đường nét đứt ngang

Nếu số dư lớn hơn >0 thì chấp nhận, ngược lại thì không thì gọi không chấp nhận

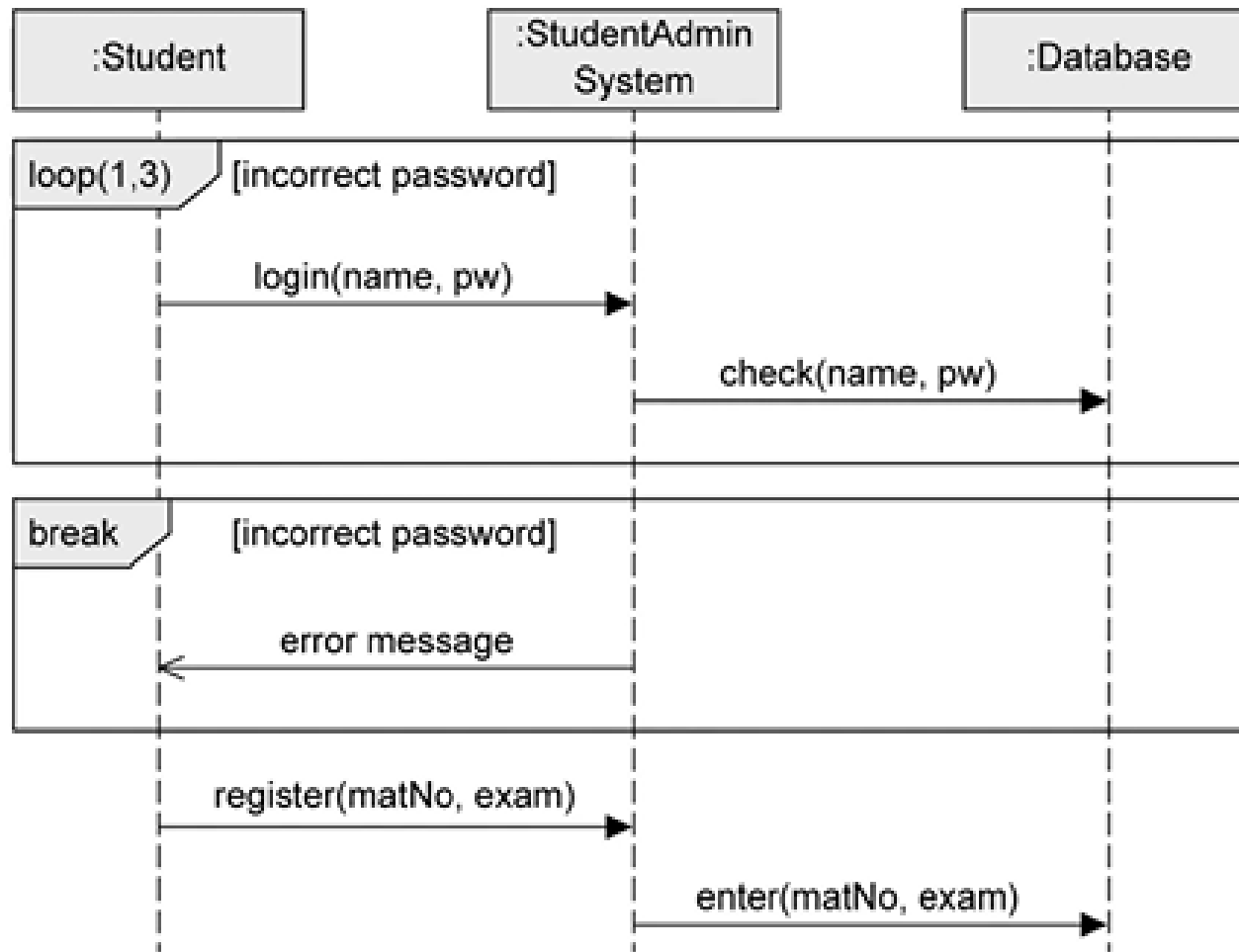
Loop

- Loop biểu diễn một vòng lặp
- Loop được theo sau bởi các số vòng lặp min và max hạn dưới bằng 0 và giới hạn trên vô hạn (min..max) hoặc (min,max), mặc định (*) là không có giới hạn trên
- Nếu vòng lặp **không có giới hạn nào được chỉ định**, điều đó có nghĩa là vòng lặp có khả năng vô hạn với giới
- Nếu chỉ xác định min - int, điều đó có nghĩa là giới hạn trên bằng giới hạn dưới và vòng lặp sẽ được thực hiện chính xác theo số lần đã chỉ định
- Nếu max - int được chỉ định, nó phải lớn hơn hoặc bằng min - int. Vòng lặp sẽ lặp lại ít nhất là min-int số lần và nhiều nhất là max - int số lần

Loop (tt)

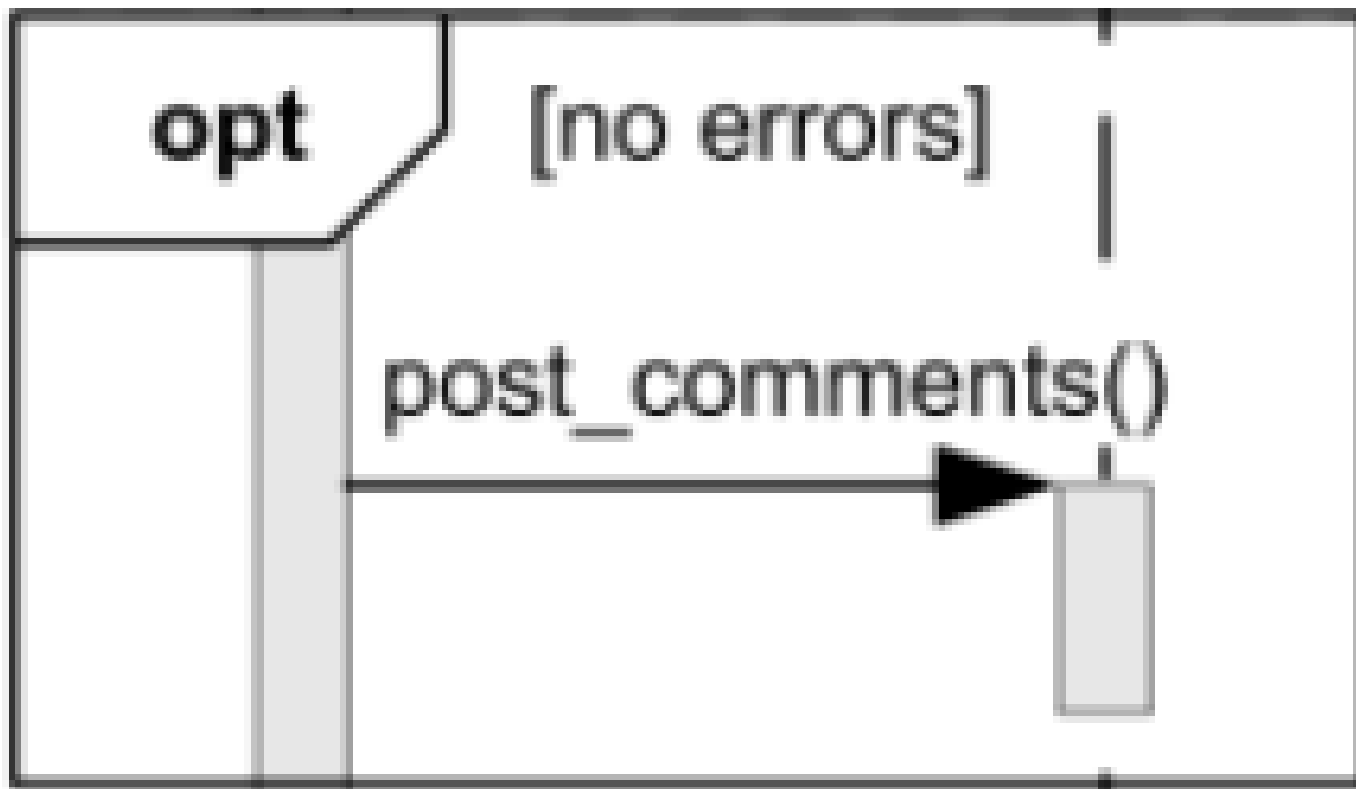


Loop (tt)



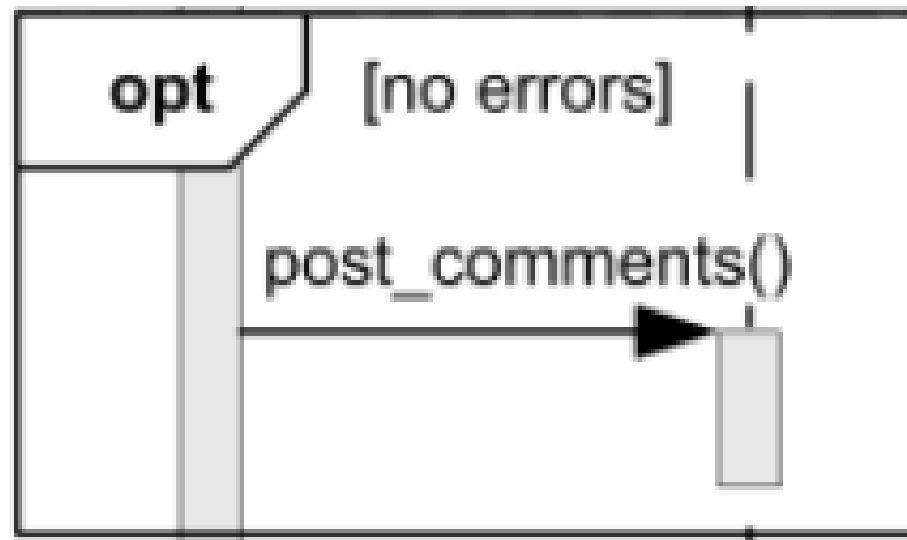
Opt

- Đại diện cho một lựa chọn hoặc lựa chọn thay thế của hành vi



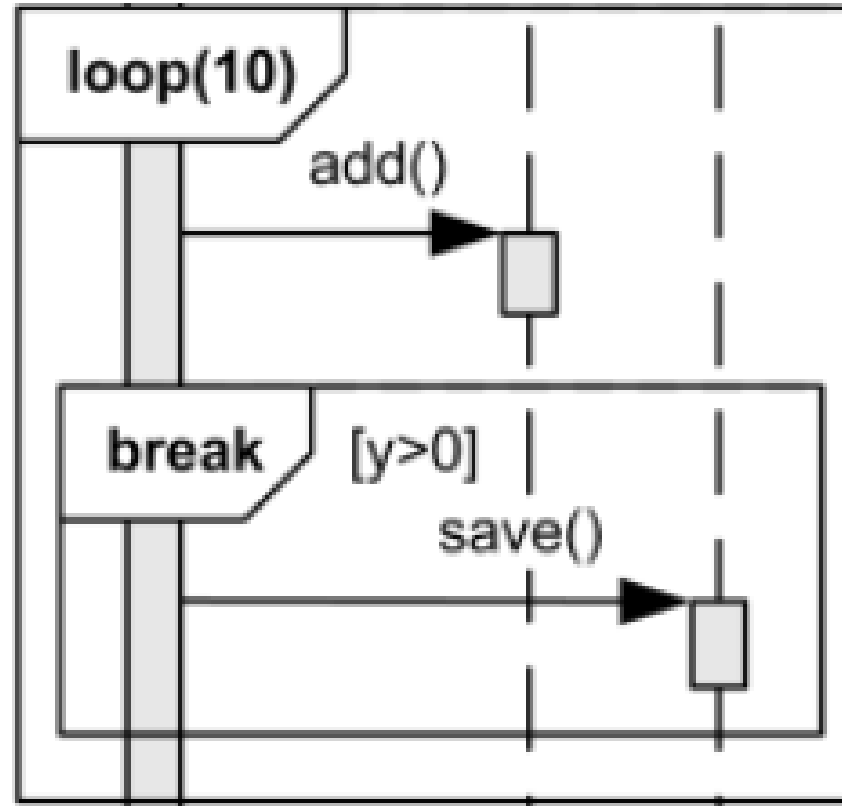
Opt (tt)

- Để mô hình hóa một chuỗi tùy chọn (**if không else**)
- Việc thực thi thực sự tại thời điểm chạy phụ thuộc vào guard
- Có chính xác một toán hạng
- Tương tự câu lệnh if mà không có nhánh else



Break

- Biểu thị một tình huống phá vỡ hoặc ngoại lệ được thực hiện thay cho phần còn lại của đoạn tương tác bao quanh



Par

- **Ý nghĩa:** Đại diện cho các đoạn tương tác được thực thi song song (đồng thời)
- **Cấu trúc:** Chứa nhiều "interaction operand", mỗi đoạn chạy độc lập với các đoạn khác
- ✓ **Ví dụ:** Trong một ứng dụng đa luồng, một đoạn xử lý dữ liệu người dùng và một đoạn khác tải hình ảnh có thể chạy song song.
- **Cách sử dụng:** Dùng khi các tác vụ có thể thực hiện đồng thời mà không phụ thuộc lẫn nhau

Neg (Negative)

- **Ý nghĩa:** Đại diện cho một kịch bản không mong muốn hoặc không hợp lệ, thường được sử dụng để minh họa các trường hợp lỗi
- **Cấu trúc:** Chứa một "interaction operand" mô tả hành vi không nên xảy ra
- ✓ **Ví dụ:** Một kịch bản [invalid login attempt] có thể được mô tả để chỉ ra hành vi không mong muốn
- **Cách sử dụng:** Dùng để kiểm tra hoặc xác định các trường hợp ngoại lệ cần tránh

Assert

- **Ý nghĩa:** Đại diện cho một khẳng định rằng chỉ có kịch bản trong "interaction operand" là hợp lệ, các kịch bản khác bị loại bỏ
- **Cấu trúc:** Chứa một "interaction operand" duy nhất, ám chỉ rằng đây là luồng duy nhất có thể xảy ra
- ✓ **Ví dụ:** [successful payment] có thể được khẳng định là kết quả duy nhất hợp lệ trong một giao dịch
- **Cách sử dụng:** Dùng để nhấn mạnh rằng không có kịch bản nào khác ngoài đoạn được mô tả

Strict

- **Ý nghĩa:** Yêu cầu các sự kiện trong "interaction operand" phải xảy ra theo thứ tự nghiêm ngặt, không cho phép xen kẽ với các sự kiện khác
- **Cấu trúc:** Chứa một "interaction operand" với thứ tự cố định
- ✓ **Ví dụ:** Trong quy trình đăng nhập, các bước [enter username] rồi [enter password] phải diễn ra theo thứ tự
- **Cách sử dụng:** Dùng khi cần đảm bảo thứ tự tuyệt đối của các tương tác

Các bước xây dựng Sequence Diagram

- Bước 1:** Xác định chức năng cần thiết kế. Dựa vào Use Case Diagram để xác định xem chức năng nào cần thiết kế
- Bước 2:** Dựa vào Activity Diagram để xác định các bước thực hiện theo nghiệp vụ
- Bước 3:** Đối chiếu với Class Diagram để xác định lớp trong hệ thống tham gia vào nghiệp vụ
- Bước 4:** Vẽ Sequence Diagram
- Bước 5:** Cập nhật lại bản vẽ Class Diagram

